CYBERSECURITY TRACK • PHP & WEB MODULE

Assignment 11

SQL Injection Defense Tool (PHP)

Done by: Abdulhadi Al-Ahdal

1. Project Overview

The objective is to build a PHP web application that queries user data from a database, simulating a search or login form[cite: 5]. The assignment compares insecure direct string concatenation (which allows SQL Injection) with secure PDO Prepared Statements that separate data from the query code[cite: 5].

2. Development Steps

1 Database Connection Setup

Initialize a database connection via PDO or set up an in-memory SQLite database for rapid testing[cite: 5].

2 Users Table Initialization

Create a users table and programmatically seed test records for verification[cite: 5].

3 Insecure Query Implementation

4 Secure Prepared Statements

Demonstrate vulnerabilities by directly appending `$_GET['username']` inside the query script[cite: 5].

Rewrite queries securely using `$pdo->prepare()` and parameter binding via `execute()` [cite: 5].

3. Technical Features & Utilities

In-Memory SQLite: Use `sqlite::memory:` for fast, standalone testing without configuring MySQL servers[cite: 5].

Error Mode Configuration: Use `PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION` for safe exception handling and to prevent sensitive disclosure[cite: 5].

4. Implementation Source Code

`sqli_defense.php`

PHP 8.x

```php
<?php
// Initialize SQLite database in memory with exception error mode
$options = [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION];
$pdo = new PDO('sqlite::memory:', null, null, $options);

// Create sample users table and insert test accounts
$pdo->exec("CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT,
    role TEXT
)");
```

```
$pdo->exec("INSERT INTO users (username, role) VALUES ('admin', 'Administrator')");
$pdo->exec("INSERT INTO users (username, role) VALUES ('alice', 'Standard User')");
$pdo->exec("INSERT INTO users (username, role) VALUES ('bob', 'Standard User')");

$inputUser = $_GET['username'] ?? 'admin';
$mode = $_GET['mode'] ?? 'secure'; // 'insecure' or 'secure'

$results = [];
$queryType = "";

if ($mode === 'insecure') {
    // VULNERABLE: Direct string concatenation allows SQL Injection
    $queryType = "Insecure (Direct Concatenation)";
    $sql = "SELECT * FROM users WHERE username = '$inputUser'";
    try {
        $stmt = $pdo->query($sql);
        $results = $stmt->fetchAll(PDO::FETCH_ASSOC);
    } catch (Exception $e) {
        $error = $e->getMessage();
    }
} else {
    // SECURE: PDO Prepared Statements separate code from data
    $queryType = "Secure (Prepared Statements)";
    $sql = "SELECT * FROM users WHERE username = ?";
    $stmt = $pdo->prepare($sql);
    $stmt->execute([$inputUser]);
    $results = $stmt->fetchAll(PDO::FETCH_ASSOC);
}

header("Content-Type: application/json; charset=UTF-8");
echo json_encode([
    "mode" => $queryType,
```

```
    "input_provided" => $inputUser,
    "records_found" => count($results),
    "data" => $results,
    "error" => $error ?? null
], JSON_PRETTY_PRINT);
?>
```

5. Acceptance Criteria

- ✓ **Injection Blocked Successfully**
  Complete mitigation of SQL Injection attempts under secure mode while retrieving correct data[cite: 5].

- ✓ **No Error or Schema Leaks**
  Absence of syntax errors or table structural leaks in secure environments[cite: 5].